

AI Intelligence Pipeline - Production Architecture

Scope: scalable ingestion, enrichment, freshness, entity resolution, resilience, and storage design for the intelligence graph workload.

1. System Overview

Sources	Ingestion	Processing	Enrichment	Storage / Delivery
Product Hunt AI products	Async aiohttp Playwright fallback	Filtering Deduplication Freshness	Gemini -> Groq -> NVIDIA DeepSeek Chunking + validation	PostgreSQL Object store Vector / graph index Google Sheet
Arxiv / Papers with Code	RSS / API crawlers	Schema normalization	LLM JSON extraction	Research Papers
5 AI news sources	Async feed crawlers	24h date gate	Optional LLM enrichment	News
5 AI job boards	Async feed/API crawlers	24h date gate	Optional LLM enrichment	Jobs

2. Scale Strategy - 500k+ Records

Horizontal workers: partition work by source, cursor/page, or hash(shard key). Workers are stateless and can scale from a laptop to a Kubernetes/VM worker pool without changing business logic.

Queue-based ingestion: publish source pages/items to a durable queue. Separate crawl, fetch, extraction, validation, and write stages so slow LLM work does not block network acquisition.

Idempotency: derive a stable fingerprint from canonical URL + source + normalized entity key. Use unique database constraints to make retries safe.

Adaptive concurrency: maintain per-host concurrency, back off on 429/5xx, and reduce concurrency when latency/error rate rises. This avoids turning scale into a rate-limit storm.

Bulk vs enrichment paths: deterministic extraction handles high-throughput fields first; expensive LLM enrichment is a second tier for unresolved records. This keeps acquisition throughput high.

3. 413 / Context Window Strategy

Risk	Mitigation
413 Payload Too Large	Pre-tokenize / estimate payload size; hard cap input characters/tokens; preserve title, headings, pricing sections, metadata, and semantically dense head/tail segments.
Long JS/HTML pages	Strip script/style/nav/footer noise before LLM submission; use a crawler-specific text budget.
Large extraction task	Split into bounded batches and aggregate validated JSON; never send thousands of products in one prompt.
Bad JSON	Strict schema validation, JSON-only prompt, retry once, then route to unresolved queue rather than hallucinating.

AI Intelligence Pipeline - Resilience & Freshness

4. 429 / Retry Strategy

Classify before retry: distinguish transient 429/503/network failures from hard quota exhaustion, invalid credentials, or insufficient balance. Hard failures disable the provider temporarily instead of wasting retries.

Backoff: exponential backoff with jitter, e.g. 1s, 2s, 4s, 8s, capped. Honor provider Retry-After / reset headers when supplied.

Provider state: maintain an in-memory/provider-state record with available flag, disabled-until timestamp, and reason. The orchestrator skips disabled providers immediately.

Fallback chain: Gemini -> Groq -> NVIDIA NIM DeepSeek. The chain is interface-compatible, so provider failures do not fail the entire batch unless all providers are unavailable.

Batch protection: when every provider is exhausted, stop or pause enrichment and persist the unresolved/infrastructure queue. Never convert API failure into a data-quality rejection.

5. Freshness Tracking and Distributed Deduplication

News/Jobs freshness: normalize published timestamps to UTC, calculate a strict now-24h cutoff, and reject older records before storage. Sources without reliable dates are quarantined for heuristic handling.

Stable content identity: canonicalize URL (scheme/host/path normalization, tracking-parameter removal where safe) and hash the resulting key. Use source + URL fingerprint as a unique key.

Distributed safety: workers acquire idempotent jobs from a queue and write through a database unique constraint/upsert. A retry of the

same article/job therefore becomes a no-op.

Incremental crawling: store last-seen cursor/time per source. Continue from the last durable checkpoint after worker failure rather than restarting the full source.

6. Anti-Bot and JS-Heavy Source Strategy

Tier 1	aiohttp / RSS / API first - cheapest and fastest path.
Tier 2	Playwright async for JavaScript-rendered pages that require browser execution.
Tier 3	Respectful throttling, per-host concurrency, caching, retries, and source-level cooldowns. Persist checkpoints so blocked sources can resume later.
Never	Do not bypass CAPTCHAs or security controls. Defer, use an official feed/API, or mark source unavailable.

7. Entity Resolution

Normalize case, punctuation, whitespace, and legal suffixes; preserve semantic tokens such as AI. Try normalized exact, compact exact, then fuzzy matching against a canonical seed list. For non-seed entities, use deterministic self-canonicalization and keep a raw->canonical mapping log.

Example: **OpenAI**, **Open AI**, and **OpenAI, Inc.** resolve to the same canonical entity while preserving the raw name for auditability.

AI Intelligence Pipeline - Storage, Operations & Delivery

8. Storage Strategy

Layer	Technology	Purpose
Primary operational store	PostgreSQL	Canonical entities, source URLs, fingerprints, timestamps, validation status, provider metadata, retry checkpoints.
Raw data / artifacts	Object storage (S3-compatible)	Raw HTML/RSS/JSON snapshots, large payloads, crawl evidence, replay/debugging.
Vector store	pgvector / managed vector DB	Semantic retrieval across news, jobs, products, and research descriptions.
Graph layer	Neo4j / PostgreSQL graph extension	Relationships: startup-product-founder-paper-job-news-source.
Delivery	Google Sheets	Human-readable final six-tab submission artifact and review surface.

9. End-to-End Production Flow

01	Source registry	URLs, API keys, feed definitions, per-source rate limits
02	Async acquisition	API/RSS/aiohttp, Playwright fallback, checkpointed pagination
03	Normalization	Canonical schema, UTC timestamps, URL normalization
04	Quality gates	Freshness, required fields, confidence, duplicate detection
05	LLM enrichment	Multi-provider extraction only for unresolved fields
06	Persistence	Upsert canonical entities + raw evidence + processing status
07	Resolution	Raw -> canonical mapping for startup/product entities
08	Delivery	JSON datasets, Google Sheet, metrics, audit logs

10. Submission Evidence from Implemented Prototype

The prototype already demonstrates asynchronous crawling, Product Hunt pagination, Playwright/aiohttp fallback, multi-tier LLM orchestration, retry classification, provider cooldown, intelligent chunking, deterministic validation, 24-hour News/Jobs freshness, and deterministic entity resolution.

The final six-tab delivery model is: Startups, Products, Research Papers, Jobs, News, Entity Mapping Log. The bulk product path is deliberately separated from the high-fidelity enrichment path so large acquisition can continue even when LLM providers are rate-limited.

Design principle: scale acquisition independently from expensive enrichment, make retries idempotent, preserve raw evidence, and never confuse infrastructure failure with data-quality failure.